Katholieke
Universiteit
Leuven

MODERN DATA ANALYTICS

Project Proposal

Group 13

Wing Fung Wingo Ng (r1082613)
Rin Yoshida (r1069065)
Dhruv Gupta (r1077995)
Manasvi Kamal Ghelani (r1077547)

3 April 2026

1 Introduction

In Flanders, bicycles are an essential mode of transportation that support people's daily lives.

The Flemish government agency, AWV (Agentschap Wegen en Verkeer), is responsible for managing regional roads, highways, and most major cycling infrastructure. In particular, maintenance planning is complex, as it must account not only for regular inspections but also for irregular interventions due to weather conditions such as heavy rain or snow.

Furthermore, maintenance work requires temporarily limiting access to roads, which directly affects bicycle traffic. For example, if maintenance is carried out during peak usage periods or when major events take place, it may lead to traffic disruption and increased inconvenience for users.

As such, AWV must make decisions under a trade-off between ensuring safety and minimizing disruption to traffic, which is inherently challenging. In particular, since bicycle usage varies significantly depending on time of day, weather, and events, it is difficult to address this complexity using conventional static planning approaches alone.

Therefore, there is a growing need for a data-driven approach that supports AWV's decision-making by taking these varying factors into account. Furthermore, these insights must be presented in an integrated manner so that AWV practitioners can intuitively understand them and make timely decisions.

To address this research question, we propose an integrated system consisting of three main components: **(i) cycling usage forecasting, (ii) web application dashboard, and (iii) data pipeline construction**.

Together, these components aim to support AWV's maintenance planning by providing a streamlined and data-driven workflow for decision-making.

2 Cycling Usage Forecasting

The Cycling usage forecasting model is the first step in solving the aforementioned gap. In addition to considering the cycling traffic data, this will require the inclusion of historical weather API (https://open-meteo.com/en/docs) for heavy rainfall, snow and even heavy winds metrics. For further deepening the use-cases of the model, we will also include top high-attendance annual events data in the Flanders region. Bike road usage particularly around the event areas is higher during these dates/timelines and will be excluded from the model to improve prediction accuracy and provide sensible maintenance interval forecasts.

To capture **seasonality based on both month, and intra-day patterns (hourly**

fluctuations), we will employ a seasonal time series model such as **SARIMA**, which explicitly models repeating temporal structures. We will benchmark this approach against a machine learning model like **XGBoost** to evaluate performance gains from nonlinear feature interactions. The execution will be based on the Python libraries of *pandas* for data processing, *statsmodels* for **SARIMA** modelling, *xgboost* for **XGBoost** modelling, and *scikit-learn* for alternative predictive models.

3 Web Application Dashboard

A web application dashboard will be built using a framework like *Streamlit*, *Dash* or *Shiny* to facilitate the maintenance scheduling workflow. The dashboard will show the underlying data and visualizations that help AWV understand the cycling traffic and weather, so that they can make better informed decisions on maintenance scheduling.

In particular, a **map visualization** using *OpenStreetMap* will be shown to display the current cycling traffic and weather data. It also features a timeline slider that allows users to change the date and time, where the map visualization will be adapted to show historical data in the past or forecasted data in the future. An **events calendar** would also be shown to display all the past maintenance events and future scheduled maintenance events, together with the estimated number of cyclists impacted.

Another feature is the **severe weather toggle button**. The toggle could be turned on when there is a severe weather condition that requires ad-hoc maintenance, such as snow or heavy rainfall. The map visualization will be adapted to show the areas that should be prioritized for maintenance based on the estimated number of cyclists impacted.

The integrated web application dashboard will serve as the main platform for AWV to get all the information they need. The streamlined workflow is going to vastly improve the efficiency of maintenance scheduling.

4 Data Pipeline Construction

The underlying data architecture will adhere to current MLOps best practices to guarantee that the maintenance planning dashboard is both resilient and dynamic. The system is designed entirely in **Python** and focuses on continuous calibration, automated orchestration, and simulated real-time data streaming.

To ensure a seamless transition to a production-based environment, all the Python scripts, ML models, and dashboards will be containerized using *Docker*. It will enable the final pipeline to be easily deployed to any cloud service (e.g. *Google Cloud Run* or *AWS Fargate*) if scaling is a requirement.

Additionally, one of the key features of the dashboard is live monitoring. To achieve this,

a mock streaming service will be engineered. A dedicated **Python script** will fetch the historical AWV data and external weather APIs, and filter for the current timestamp. This script will push the data row by row into a lightweight relational database (*PostgreSQL*, *SQLite*, *InfluxDB*) at regular intervals, which will enable the web application to query through it and create a highly realistic simulation of live traffic and weather feed.

Furthermore, because of the drift impact through cycling and weather behaviors, the forecasting model cannot remain static. For this, an automated workflow of model registry and continuous calibration will be implemented. *MLFlow* will be integrated to track the experiments, parameters, evaluation metrics, and some artifacts of every trained model. To execute the weekly calibration, a task orchestrator (*Prefect*, *CronJob*) will be used. This can help trigger the script which fetches the latest simulation data, retrains the forecasting model, logs the performance metrics in *MLFlow*, and automatically promotes the new model to production if it outperforms the previous version.

The resulting architecture will help the dashboard to remain automated, lightweight, responsive, and consistently updated with both current and future predictions.